About Parameters and Parameter Files

ECJ relies heavily on parameter files for nearly *every* conceivable parameter setting. It even relies on parameter files to determine which classes to use in different places. This means that understanding parameters and parameter files is crucial to using ECJ.

Parameters

ECJ's parameters are written one to a line in Java property-list style. They may be in one of the two following formats:

```
parametername = value
parametername value
```

The second option is deprecated, please don't do it. Whitespace is stripped. Parameter values may contain internal whitespace but parameter names may not. Blank lines and lines beginning with a "#" are ignored. Parameter names and values are **case-sensitive**.

Parameter values are interpreted as one of five data types, depending on the parameter:

- **Strings**. Consist of everything after the equals sign and before the newline, trimmed of whitespace.
- **Classnames**. Parsed as Strings are. Must be the full absolute classname (for example, `java.util.Hashtable`).
- **Pathnames**. Parsed as Strings are. Pathnames can be either absolute (as in /foo/bar) or relative (as in foo/bar or ../../foo/bar). If they are relative, they are interpreted relative to the directory of the parameter file in which they are found. However, if a relative pathname is prefixed with a "$" (as in $foo/bar or $../../foo/bar), then it is assumed relative with respect to the directory in which the java process was started.
- **Integers and Floating-point Numbers**. Must be in Java-standard numerical form.
- **Booleans**. Boolean values are of the form:

  ```
  parametername = true
  parametername = false
  ```

  If a parameter is declared but is not one of these two values, it is assumed to be a "default" value, which varies depending on the parameter. For example, if the default for "myparameter" is `true`, then:

  ```
  myparameter = gobbledygook
  myparameter =
  ```

  ...both signify that myparameter is to be set to "true".

Parameter Files

ECJ reads parameters from a hierarchical set of parameter files, typically called "params" or ending with the extension ".params". When you start up ECJ, you specify a parameter file as such:

```
java ec.Evolve -file myParameterFile
```

Parameter files can have multiple parents which define additional parameters. A parameter file specifies that it has a parent with a special parameter:

```
parent.n = parentFile
```

...where *n* indicates that the parent is parent #*n*. *n* starts at 0 and increases. Your parents must be assigned with consecutive parameter names starting with `parent.0`. For example:

```
parent.0 = ../../myFirstParent.params
parent.1 = ../../../mySecondParent.params
parent.2 = ../foo/bar/myThirdParent.params
```

Precedence

Parameters may also be defined on the command line when running ECJ with the "-p" option, which may appear multiple times. No space may appear between the parameter name, "=", and value. For example:

```
java ec.Evolve -file my.params -p extraparam=extravalue -p anotherparam=anothervalue
```

Parameters may further be programatically defined internally by the system, though ECJ presently never does this. If you have two parameters with the same name, here are the rules guiding which ones take precedence:

- Programmatically-set parameters override all others.
- Next come command-line parameters.
- Later command-line parameters override earlier ones.
- Next come parameters in the specified parameter file.
- Within a file, later parameters override earlier ones.
- Child parameter files' parameters override their parents' or ancestors' parameters.
- If a file has parents *parent.n* and *parent.m*, then all parameters derived from *parent.n* and its ancestors override all parameters derived from *parent.m* and its ancestors if *n* is less than *m*.

ECJ's Parameter Style

Since numerous objects read parameters from the parameter database, ECJ organizes its parameter namespace hierarchically using periods to separate elements in parameter names. Let's begin with the simplest situation: someECJ parameters are simple global parameters. For example,

```
evalthreads = 4
```

...tells ECJ that it should spawn 4 threads when doing population evaluation. Other parameters are organized hierarchically because it's cleaner that way. For example, if *evalthreads* and *breedthreads* are both 4, then there are 4 seeds for the random number generator which must be defined. They are defined as such (Note the period between *seed* and the number *n*):

```
seed.0 = 2341
seed.1 = 7234123
seed.2 = 411
seed.3 = 34021239
```

It's common for arrays of objects are defined like this, with numbers representing their position.

The period is used for other hierarchical purposes. When an object contains other objects as subordinates, they fall within its hierarchy. Such objects have a *parameter base* which is prefixed to them. For example, the global Population instance contains an array of Subpopulation instances, each of which in turn contain a variety of objects. Here's how the Population instance is defined, the number of subpops it contains is set, the classes for its various subpopulations are defined, and the number of individuals each one has is set:

```
# We're doing some coevolution, so we need two
# subpopulations, each with 500 individuals
pop = ec.Population
pop.subpops = 2
pop.subpop.0 = ec.Subpopulation
```

```
pop.subpop.0.size = 500
pop.subpop.1 = ec.Subpopulation
pop.subpop.1.size = 500
```

Note that the parameters for each subpopulation begin with the parameter base `pop.subpop.`*`n`*. Each Subpopulation instance requests a "size" relative to its current parameter base handed it by its "controlling" object. As you might guess, these hierarchical bases can get very long.

If an object needs a given parameter, and the parameter does not exist with the provided base, then the object can check a *default base* for the parameter. For example, let's say that breeding pipeline #0 of the species for subpopulation #1 of the population is a MutationPipeline (GP point mutation) and is using Tournament Selection as it's source #0 to select individuals. It might declare some information thusly:

```
pop.subpop.1.species.pipe.0 = ec.gp.koza.MutationPipeline

pop.subpop.1.species.pipe.0.source.0 = ec.select.TournamentSelection
```

...we can custom-define the tournament size parameter by tacking it onto this base as:

```
pop.subpop.1.species.pipe.0.source.0.size = 7
```

...or we can fall back on a "default" setting for this parameter for all Tournament Selection objects as:

```
select.tournament.size = 2
```

...In this case the hierarchical parameter base is `pop.subpop.1.species.pipe.0.source.0` and the "default base" for Tournament Selection is `select.tournament`. If the object looks both places and still can't find a parameter defined (or it's improperly defined), it will issue an error. Some global objects don't have default parameter bases, but most every object which can be repeatedly declared in different places will have a default base.

In general, objects which read parameters fall into one of several classes:

- **Singletons** are high-level global objects, and typically have global parameters or form the root of parameter hierarchies. For example, the EvolutionState object is a Singleton.
- **Groups** (Populations and their subpopulations) are rooted globally with `pop`.
- **Cliques** are small groups of instances which together form a global family. For example, GP Types are a clique. Typically Cliques form their own little hierarchies rooted globally.
- **Prototypes** hang off of Singletons and Groups, and are never the root of a hierarchy. Prototypes almost always have default bases. For example, Tournament Selection objects are Prototypes.

Tracing Bases Through Class Documentation

The class documentation contains three tables which give information about parameters and parameter bases for instances of that class. The **Parameters** table indicates the valid parameters declared for that instance. The **Default Base** indicates the class's default base, if any. The **Parameter Bases** table indicates the new parameter bases for subsidiary objects to this instance. For example, here's the tables from ec.gp.koza.MutationPipeline, the class responsible for doing the GP point mutation operator:

Parameters

base.`tries` int >= 1	(number of times to try finding valid pairs of nodes)
base.`maxdepth` int >= 1	(maximum valid depth of a crossed-over subtree)
base.`ns` classname, inherits and != GPNodeSelector	(GPNodeSelector for tree)

base.`build`.0 classname, inherits and != GPNodeBuilder	(GPNodeBuilder for new subtree)
`equal` bool = `true` or `false` (default)	(do we attempt to replace the subtree with a new one of roughly the same size?)
base.`tree.0` 0 < int < (num trees in individuals), if exists	(tree chosen for mutation; if parameter doesn't exist, tree is picked at random)

Default Base
gp.koza.mutate

Parameter bases
base.`ns` nodeselect
base.`build` builder

MutationPipeline is derived from **ec.BreedingPipeline**, which adds the following tables:

Parameters

base.`num-sources` int >= 1	(User-specified number of sources to the pipeline. Some pipelines have hard-coded numbers of sources; others indicate (with the java constant DYNAMIC_SOURCES) that the number of sources is determined by this user parameter instead.)
base.`source.`*n* classname, inherits and != BreedingSource, or the value `same`	(Source *n* for this BreedingPipeline. If the value is set to `same`, then this source is the exact same source object as *base*.`source.`*n-1*, and further parameters for this object will be ignored and treated as the same as those for *n-1*. `same is not valid for` *base*`.source.0`)

Parameter bases
base.`source.`*n* Source *n*

ec.BreedingPipeline in turn is derived from **ec.BreedingSource**, which adds the following tables:

Parameters

base.`prob` 0.0 <= float <= 1.0, or undefined	(probability this BreedingSource gets chosen. Undefined is only valid if the caller of this BreedingSource doesn't need a probability)

Although MutationPipeline inherits all these parameters, the parameter base for *all* of them is the instance's parameter base handed it by its controller object. And the default base for all of them is always the last one defined (in this case, "gp.koza.mutate". Default bases for parent classes are not used.

Back to our original example, imagine that we had a MutationPipeline used as breeding pipeline #0 of the species used in subpopulation #1 of the population:

```
pop.subpop.1.species.pipe.0 = ec.gp.koza.MutationPipeline
```

We could specify a probability for this pipeline as:

```
pop.subpop.1.species.pipe.0.prob = 0.9
```

...or we might specify a default probability (not necessarily a good idea) for all MutationPipelines as:

```
gp.koza.mutate.prob = 0.4
```

MutationPipeline contains two subsidiary instances, one which subclasses from **gp.GPNodeSelector**, and one which subclasses from **gp.GPNodeBuilder**. The first is responsible for picking a subtree to mutate, and the second is responsible for creating a new subtree. We specify classes for those instances in their parameters (we'll use a KozaNodeSelector and a GrowBuilder):

```
pop.subpop.1.species.pipe.0.ns.0 = ec.gp.koza.KozaNodeSelector
pop.subpop.1.species.pipe.0.build.0 = ec.gp.koza.GrowBuilder
```

Of course, we might provide default choices as well:

```
gp.koza.mutate.ns.0 = ec.gp.koza.KozaNodeSelector
gp.koza.mutate.build.0 = ec.gp.koza.GrowBuilder
```

These two objects have parameters to set up as well. Their parameter bases are specified as `base.ns` and `base.build` respectively. In this case, it means that their parameter bases are `pop.subpop.1.species.pipe.0.ns.0` and `pop.subpop.1.species.pipe.0.build.0`. And thus the cycle of life continues. For example, KozaNodeSelectors have default base of `gp.koza.ns` and a `root` parameter which specifies the probability they'd pick the root of a tree. The root parameter would then be found at `pop.subpop.1.species.pipe.0.ns.0.root`, or the default value at `gp.koza.ns.root`.

Where to look for specifics about parameters

There are way too many possible parameters to discuss here. Here are some places to start digging.

- ec/params
- ec/simple/params and ec/gp/koza/params if you're doing GP
- different problem's parameter files
- ec.Evolve contains documentation on many basic parameters
- ec.EvolutionState holds most subsidiary objects.
- ec.Population is the root object for a lot of parameter bases.

Parameters currently used by Symbolic Regression

Some are global parameters, some are defined through the parameter base hierarchy, and some are defined through default bases. The parameter files are app/regression/noerc.params, its parent gp/koza/params, and its parent simple/params.

```
Number of threads and random number generator seeds
breedthreads = 1
evalthreads = 1
seed.0 = 4357

Garbage collection
gc = false
aggressive = true
gc-modulo = 1

Checkpointing
checkpoint = false
checkpoint-modulo = 1
prefix = ec

Outputting Stuff
nostore = false
flush = true
verbosity = 0

The EvolutionState Object
```

```
state = ec.simple.SimpleEvolutionState
```

Evolution Parameters

```
generations = 51
quit-on-run-complete = true
```

The Initializer, Breeder, Exchanger, and Finisher

```
breed = ec.simple.SimpleBreeder
exch = ec.simple.SimpleExchanger
finish = ec.simple.SimpleFinisher
init = ec.gp.GPInitializer
```

The Evaluator and the Problem (ADF stuff is always loaded but not used in this case)

```
eval = ec.simple.SimpleEvaluator
eval.problem = ec.app.regression.Regression
eval.problem.data = ec.app.regression.RegressionData
eval.problem.stack = ec.gp.ADFStack
eval.problem.stack.context = ec.gp.ADFContext
eval.problem.stack.context.data = ec.app.regression.RegressionData
```

The Statistics

```
stat = ec.gp.koza.KozaStatistics
stat.file = $out.stat
```

Default Tournament Selection tournament size

```
select.tournament.size = 7
```

Default HalfBuilder (ramped half/half tree building) parameters

```
gp.koza.half.growp = 0.5
gp.koza.half.max-depth = 6
```

Default KozaNodeSelector parameters

```
gp.koza.ns.nonterminals = 0.9
gp.koza.ns.root = 0.0
gp.koza.ns.terminals = 0.1
```

Default Reproduction operator parameters

```
gp.koza.reproduce.source.0 = ec.select.TournamentSelection
```

Default Crossover operator parameters

```
gp.koza.xover.maxdepth = 17
gp.koza.xover.ns.0 = ec.gp.koza.KozaNodeSelector
gp.koza.xover.ns.1 = same
gp.koza.xover.source.0 = ec.select.TournamentSelection
gp.koza.xover.source.1 = same
gp.koza.xover.tries = 1
```

Function Sets (there's only one)

```
gp.fs.size = 1
gp.fs.0 = ec.gp.GPFunctionSet
gp.fs.0.name = f0
gp.fs.0.size = 9
gp.fs.0.func.0 = ec.app.regression.func.X
gp.fs.0.func.0.nc = nc0
gp.fs.0.func.1 = ec.app.regression.func.Add
gp.fs.0.func.1.nc = nc2
gp.fs.0.func.2 = ec.app.regression.func.Mul
gp.fs.0.func.2.nc = nc2
gp.fs.0.func.3 = ec.app.regression.func.Sub
gp.fs.0.func.3.nc = nc2
gp.fs.0.func.4 = ec.app.regression.func.Div
gp.fs.0.func.4.nc = nc2
gp.fs.0.func.5 = ec.app.regression.func.Sin
gp.fs.0.func.5.nc = nc1
gp.fs.0.func.6 = ec.app.regression.func.Cos
```

```
gp.fs.0.func.6.nc = nc1
gp.fs.0.func.7 = ec.app.regression.func.Exp
gp.fs.0.func.7.nc = nc1
gp.fs.0.func.8 = ec.app.regression.func.Log
gp.fs.0.func.8.nc = nc1
```

Standard Node Constraints for untyped GP with nodes of various arity sizes

```
gp.nc.size = 7
gp.nc.0 = ec.gp.GPNodeConstraints
gp.nc.0.name = nc0
gp.nc.0.returns = nil
gp.nc.0.size = 0
gp.nc.1 = ec.gp.GPNodeConstraints
gp.nc.1.name = nc1
gp.nc.1.returns = nil
gp.nc.1.size = 1
gp.nc.1.child.0 = nil
gp.nc.2 = ec.gp.GPNodeConstraints
gp.nc.2.name = nc2
gp.nc.2.returns = nil
gp.nc.2.size = 2
gp.nc.2.child.0 = nil
gp.nc.2.child.1 = nil
gp.nc.3 = ec.gp.GPNodeConstraints
gp.nc.3.name = nc3
gp.nc.3.returns = nil
gp.nc.3.size = 3
gp.nc.3.child.0 = nil
gp.nc.3.child.1 = nil
gp.nc.3.child.2 = nil
gp.nc.4 = ec.gp.GPNodeConstraints
gp.nc.4.name = nc4
gp.nc.4.returns = nil
gp.nc.4.size = 4
gp.nc.4.child.0 = nil
gp.nc.4.child.1 = nil
gp.nc.4.child.2 = nil
gp.nc.4.child.3 = nil
gp.nc.5 = ec.gp.GPNodeConstraints
gp.nc.5.name = nc5
gp.nc.5.returns = nil
gp.nc.5.size = 5
gp.nc.5.child.0 = nil
gp.nc.5.child.1 = nil
gp.nc.5.child.2 = nil
gp.nc.5.child.3 = nil
gp.nc.5.child.4 = nil
gp.nc.6 = ec.gp.GPNodeConstraints
gp.nc.6.name = nc6
gp.nc.6.returns = nil
gp.nc.6.size = 6
gp.nc.6.child.0 = nil
gp.nc.6.child.1 = nil
gp.nc.6.child.2 = nil
gp.nc.6.child.3 = nil
gp.nc.6.child.4 = nil
gp.nc.6.child.5 = nil
```

Tree Constraints

```
gp.tc.size = 1
gp.tc.0 = ec.gp.GPTreeConstraints
gp.tc.0.init = ec.gp.koza.HalfBuilder
gp.tc.0.name = tc0
gp.tc.0.returns = nil
```

GP Types

```
gp.type.a.size = 1
gp.type.a.0.name = nil
gp.type.s.size = 0
```

The Population, and its one subpopulation, species, breeding pipelines and individuals

```
pop = ec.Population
pop.subpops = 1
pop.subpop.0 = ec.Subpopulation
pop.subpop.0.duplicate-retries = 100
pop.subpop.0.fitness = ec.gp.koza.KozaFitness
pop.subpop.0.size = 1000
pop.subpop.0.species = ec.gp.GPSpecies
pop.subpop.0.species.ind = ec.gp.GPIndividual
pop.subpop.0.species.ind.numtrees = 1
pop.subpop.0.species.ind.tree.0 = ec.gp.GPTree
pop.subpop.0.species.ind.tree.0.tc = tc0
pop.subpop.0.species.numpipes = 2
pop.subpop.0.species.pipe.0 = ec.gp.koza.CrossoverPipeline
pop.subpop.0.species.pipe.0.prob = 0.9
pop.subpop.0.species.pipe.1 = ec.gp.koza.ReproductionPipeline
pop.subpop.0.species.pipe.1.prob = 0.1
```